

SIMATS ENGINEERING

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	S Susmitha	Reg.No.:	192571068
Department:	BTech CSE BioSciences	Date:	20.08.2026

ANNEXURE A INDUSTRY PROBLEM SOLVING TASK

- Smartphone Performance Optimization**
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
- Cloud Server Memory Bottleneck**
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.
- Autonomous Vehicle Data Processing**
An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.
- AI Inference Edge Device**
An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.
- High-Performance Gaming Console**
A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct:

*/ S Susmitha (192571068) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Name	Register Number	Course Code	Course
S Susmitha	192571068	CSA1202	Computer Architecture

1. Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. This points to frequent cache misses and DRAM refresh delays as the phone's working set changes with every app switch.

Q1: Smartphone Memory Hierarchy (fast/small → slow/large)

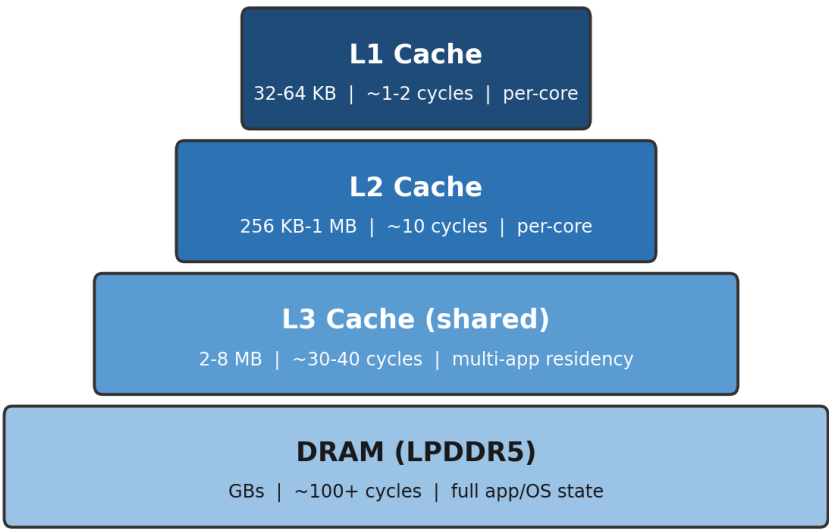


Fig. 1.1 – Smartphone memory hierarchy: narrower/faster levels sit above larger/slower ones.

Level	Typical Size	Latency	Role in Redesign
L1 Cache	32–64 KB	~1–2 cycles	Immediate data/instructions for the active app
L2 Cache	256 KB–1 MB	~10 cycles	Per-core working set retention
L3 Cache	2–8 MB (shared)	~30–40 cycles	Multi-app residency; primary target of redesign

Level	Typical Size	Latency	Role in Redesign
DRAM (LPDDR5)	GB-scale	~100+ cycles	Full app/OS state; traffic to be minimized

Problem Analysis

App switching forces the processor to evict cache lines belonging to the previous app and reload data for the new one. Since mobile SoCs run several background processes (launcher, notification services, the new foreground app), the L1 and L2 caches are repeatedly flushed, and DRAM has to be accessed far more often than in a steady single-app workload. Each DRAM access costs roughly 100–150 processor cycles compared to 1–4 cycles for L1 and about 10–20 cycles for L2, so this thrashing is the direct source of visible lag.

Role of the Cache Hierarchy

- L1 cache (32–64 KB, split I/D, ~1–2 cycle latency): holds the instruction stream and immediate data of the app currently in focus; too small to retain context across a switch.
- L2 cache (256 KB–1 MB, unified, ~10 cycle latency): buffers per-core working sets and can retain some recently used app data if sized generously.
- L3 cache (2–8 MB, shared across cores, ~30–40 cycle latency): is the key resource for multitasking because it can hold state for more than one app at once, reducing the number of full DRAM round-trips during a switch.
- DRAM (LPDDR5, ~100+ cycle latency): backs everything but is power-hungry and slow relative to on-chip cache, so the design goal is to keep switch-time traffic out of DRAM as far as possible.

Proposed Hierarchical Redesign

- Increase shared L3 capacity so the two or three most recently used apps' working sets survive a switch without eviction to DRAM.
- Adopt a victim cache between L2 and L3 to catch lines evicted during a switch instead of discarding them immediately.
- Use way-prediction / prefetching tuned to the app-launch pattern so instruction fetch for the newly foregrounded app starts before the user interaction completes.
- Pair LPDDR5X DRAM with fast idle-to-active transition modes so that when L3 does miss, the DRAM penalty is minimized.

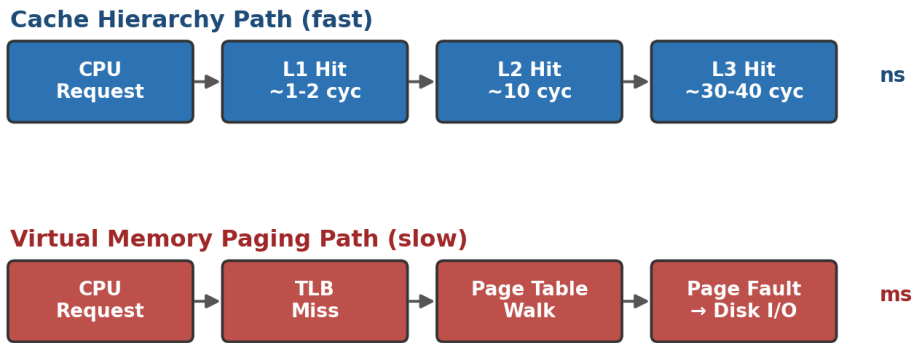
Expected Outcome

Sizing L3 for multi-app residency and adding a victim cache converts many DRAM accesses into L3 hits, cutting effective memory latency during switching and directly reducing perceived lag while keeping power draw in check.

2. Cloud Server Memory Bottleneck

A cloud provider is seeing high latency in database queries. The two candidate causes are an inadequate cache hierarchy and inefficient virtual memory paging, and the two need to be compared rather than treated as the same problem.

Q2: Cache-Hit Path vs Page-Fault Path (Cloud Database Server)



Paging penalty is ~5-6 orders of magnitude larger than a cache miss

Fig. 2.1 – Cache-hit path (nanosecond scale) versus page-fault path (millisecond scale).

Aspect	Cache Hierarchy	Virtual Memory Paging
Latency scale	Nanosecond level	Millisecond level (on disk-backed fault)
Root cause here	Working set exceeds L2/L3 capacity	Physical RAM undersized for buffer pool
Primary fix	Better locality, prefetching, larger L3	More RAM, huge pages, larger TLB reach

Cache Hierarchy Behaviour

Database workloads have large, often random-access working sets (indexes, buffer pools) that regularly exceed L2 and even L3 capacity. When the working set does not fit in cache, the hierarchy cannot help beyond a point, and every miss falls through to DRAM or, worse, to disk-backed pages.

Virtual Memory Paging Behaviour

Virtual memory allows the database's address space to exceed physical RAM, but each TLB miss requires a page-table walk (multiple memory accesses), and each page fault that goes to secondary storage costs on the order of milliseconds — roughly five to six orders of magnitude slower than a cache hit. If the server's physical RAM is undersized for the buffer pool, the system will page frequently, and this dominates latency far more than any cache effect.

Comparison

- Cache hierarchy: microsecond-to-nanosecond level impact; solved by better locality, prefetching, larger L3.
- Virtual memory paging: millisecond-level impact when it spills to disk; solved by more physical RAM, larger TLB reach, and huge pages.
- A cache-hierarchy fix cannot compensate for a paging bottleneck: no amount of L3 tuning helps if pages are being swapped to disk, so root-causing which layer is actually thrashing (via miss-rate and page-fault counters) must come first.

Recommended Improvements

- Increase physical DRAM to keep the active buffer pool resident, cutting page-fault rate.
- Enable huge pages (2 MB/1 GB) to reduce TLB miss frequency for large sequential scans and index structures.
- Use NVDIMM or DRAM-backed persistent memory for write-ahead logs so log flushes do not compete with buffer-pool DRAM.
- Tune L3 cache partitioning (e.g., Intel CAT/AMD equivalent) so the database process is not evicted by co-located tenant workloads.

Expected Outcome

Addressing the paging layer first removes the millisecond-scale stalls, after which cache-level tuning yields further, smaller latency gains.

3. Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data (LiDAR, radar, camera) in real time, which demands both very high bandwidth for streaming sensor input and very low, predictable latency for control-loop decisions.

Q3: Autonomous Vehicle Memory Roles by Technology

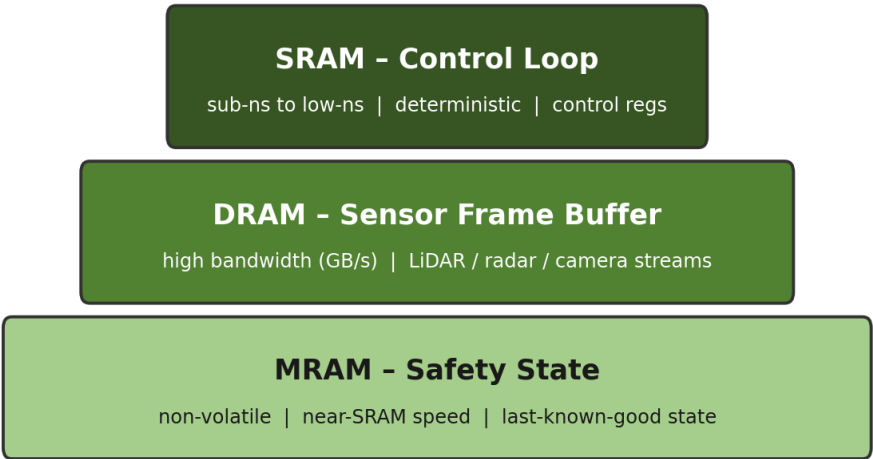


Fig. 3.1 – Mapping of SRAM, DRAM and MRAM to distinct roles in the vehicle's memory hierarchy.

Technology	Latency	Volatility	Role in Vehicle
SRAM	Sub-ns to low-ns	Volatile	Control-loop registers, deterministic access
DRAM	Tens of ns	Volatile	High-bandwidth sensor frame buffer
MRAM	Near-SRAM speed	Non-volatile	Safety-critical state, survives power loss

Technology Evaluation

- SRAM: sub-nanosecond to low-nanosecond access, no refresh needed, but low density and high per-bit cost — suited only to small, latency-critical buffers such as L1/L2 cache and real-time control registers.
- DRAM: much higher density at lower cost per bit, but needs periodic refresh and has higher, less predictable latency — suited to bulk frame buffers holding raw sensor data before processing.
- MRAM: non-volatile, near-SRAM speed, unlimited endurance, and no refresh power overhead — well suited to storing safety-critical state (last-known-good trajectory, sensor calibration) that must survive a power glitch without a write-cycle wear penalty.

Proposed Memory Hierarchy

- Tier 1 – SRAM scratchpad/cache tightly coupled to the perception and control cores for deterministic, cycle-accurate access to the current sensor frame and control variables.
- Tier 2 – High-bandwidth DRAM (e.g., GDDR6/HBM-class) as the shared frame buffer for LiDAR point clouds and camera streams, since this data is bandwidth-bound rather than latency-bound.
- Tier 3 – MRAM for non-volatile retention of safety-state and configuration data, so a power interruption never loses the vehicle's last valid decision context.

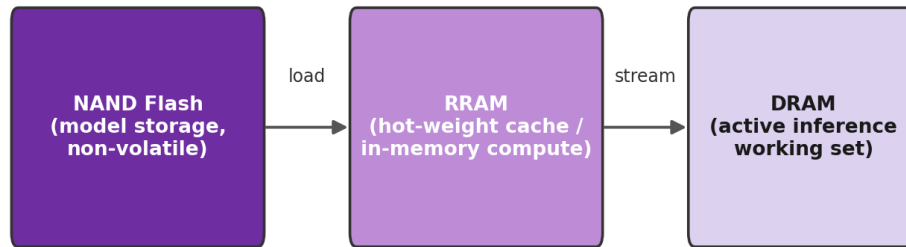
Analysis

This tiering matches each technology's strength to its role: SRAM guarantees the microsecond-level determinism that a control loop needs, high-bandwidth DRAM absorbs the sustained gigabyte-per-second sensor throughput, and MRAM removes the single point of failure that a pure-DRAM design would have on power loss. The result maximizes both bandwidth (via DRAM) and minimizes worst-case latency (via SRAM) while adding fail-safe persistence.

4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption, so the trade-off here is between non-volatility, speed, and energy per access rather than raw bandwidth alone.

Q4: AI Edge Device - Model Load & Inference Data Path



Frequently reused weights served from RRAM instead of re-reading NAND → faster load, lower power

Fig. 4.1 – Model weights flow from persistent NAND storage through an RRAM hot-weight cache into DRAM working memory.

Technology	Latency	Volatility	Role in Edge Device
NAND Flash	Tens of μ s	Non-volatile	Persistent, high-density model storage
DRAM	Nanosecond level	Volatile	Active layer/activation working set
RRAM	Low latency, low energy	Non-volatile	Hot-weight cache; supports in-memory compute

Technology Evaluation

- NAND Flash: non-volatile and dense, so it is the natural place to store the trained model weights, but read latency (tens of microseconds) and limited write endurance make it unsuitable for runtime working memory.
- DRAM: fast enough (nanosecond-level) to hold the model's active layers and intermediate activations during inference, but volatile and comparatively power-hungry, especially due to constant refresh.
- RRAM: non-volatile like NAND but with much lower read latency and lower energy per access, and it also supports in-memory computation (analog multiply-accumulate), which is attractive for low-power neural inference.

Optimized Hierarchical Structure

- Store the trained model persistently in NAND Flash for cost-effective, high-density storage.

- Load the model into a DRAM working set for the active inference session, since DRAM's speed suits the repeated read/write pattern of layer computation.
- Introduce an RRAM-based cache/compute layer between Flash and DRAM: it can hold frequently reused weights (e.g., early convolutional layers) with near-instant, low-power access, and if RRAM-based compute-in-memory is used, some multiply-accumulate operations can be offloaded to it directly, cutting data movement to the digital core.

Analysis

This hierarchy meets the "fast model loading" requirement by letting RRAM shortcut the slow NAND read path for the model's most-used weights, while DRAM continues to handle transient activation data. Because RRAM and NAND are both non-volatile, the device also avoids the energy cost of repeatedly reloading the full model from Flash into DRAM on every wake cycle, which is where most of the power saving comes from.

5. High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading, which indicates that streaming asset data from memory cannot keep pace with the GPU's rendering throughput at that moment.

Q5: L3 Cache vs DRAM - Bandwidth, Capacity and Latency

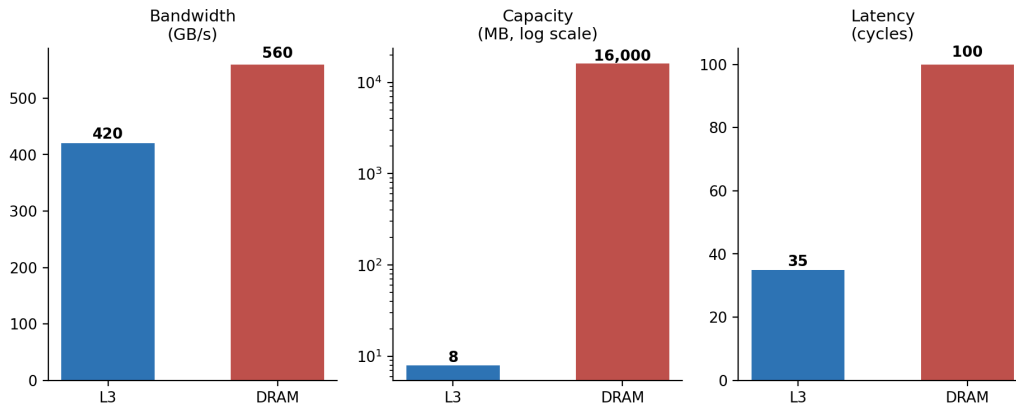


Fig. 5.1 – L3 cache vs DRAM: bandwidth, capacity, and latency (illustrative console-class values).

Metric	L3 Cache	DRAM (GDDR6)
Bandwidth	~420 GB/s	~560 GB/s
Capacity	~8 MB	~16,000 MB (16 GB)
Latency	~35 cycles	~100 cycles

L3 Cache vs DRAM Throughput

L3 cache offers very high bandwidth (hundreds of GB/s) but limited capacity (a few MB to tens of MB), so it can service the GPU's immediate texture and geometry accesses only for the portion of a scene already resident. When a new scene loads, the required assets (textures, meshes, level geometry) usually exceed L3 capacity by orders of magnitude, forcing the GPU to pull directly from DRAM. Console DRAM (GDDR6, ~500+ GB/s) is fast relative to typical system DRAM, but a single burst of asset streaming can still saturate available bandwidth, and the latency of the first miss on each new asset is unmasked because there is no cached copy to fall back on — this burst is what shows up as a frame drop.

Proposed Enhancements

- Increase effective L3 capacity or add a dedicated GPU-side last-level cache scoped to texture/geometry streaming, so more of an incoming scene's working set can be pre-resident.
- Use asynchronous asset prefetching during the preceding scene (predictive streaming) so DRAM bandwidth demand is spread over time instead of spiking at the load boundary.
- Employ compressed texture formats and on-the-fly decompression close to the cache, reducing the volume of data that must cross the DRAM bus per frame.

- Use a unified memory architecture with high-bandwidth GDDR6/GDDR6X so CPU and GPU are not contending for separate, narrower memory pools during the load spike.

Analysis

Because the frame-drop event is bandwidth-bound and burst-driven rather than steady-state, the highest-value fix is temporal: spreading DRAM traffic earlier via prefetching reduces the peak demand more effectively than adding cache capacity alone, though the two measures are complementary and are recommended together.